

chat vc pode me ajudar com relatório de paradigmas? linguagem da vez é C#, orientação a objetos

ai n me de as respostas, apeans ajude a pensar ok;?

Claro, Caio! Vamos fazer em **C# com orientação a objetos**, e eu vou te ajudar a pensar, com perguntas e dicas, sem entregar as respostas nem o código pronto.

Manda o primeiro exercício e o que você já entendeu dele. Vamos por partes, da maneira mais simples possível 😊

qual a diferença entre uma propriedade e um atributo em c#?

Em C#, o que normalmente chamamos de **atributo** em POO é um **campo**: uma variável dentro da classe que guarda um valor.

Uma **propriedade** permite ler ou alterar esse valor usando `get` e `set`.

</> C#

```
class pessoa
{
    private int idade;

    public int Idade
    {
        get { return idade; }
        set { idade = value; }
    }
}
```

Nesse exemplo:

- `idade` é o **campo** que armazena o valor.
- `Idade` é a **propriedade** que dá acesso a ele.
- `get` retorna o valor.
- `set` recebe o novo valor, representado por `value`.

A propriedade permite colocar regras nesse acesso, como impedir uma idade negativa.

Pensando nisso: **em qual parte você colocaria essa verificação, no `get` ou no `set` ?**

como faco um construtor receber nome, povo e posto?

O construtor precisa ter **o mesmo nome da classe**, não tem tipo de retorno e recebe os dados entre parênteses.

Por exemplo, só para entender a estrutura:

</> C#

```
public NomeDaClasse(string nome, string povo, string posto)
{

}
```

Use `string` se os três dados forem textos.

Dentro dele, você precisa guardar cada parâmetro no campo ou na propriedade correspondente do objeto. Quando o campo e o parâmetro têm o mesmo nome, `this.nome` identifica o campo do objeto, enquanto `nome` identifica o parâmetro.

Como se chama sua classe e como você declarou esses três dados nela?

como uma classe herda de outra em c#?

Em C#, você usa **dois-pontos** : depois do nome da classe:

</> C#

```
class aluno : pessoa
{

}
```

Isso significa que **aluno herda de pessoa**. `pessoa` é a classe base e `aluno` é a classe derivada.

Diferente do C++, você não coloca `public` depois dos dois-pontos.

Os membros `public` e `protected` da base podem ser acessados pela classe derivada. Os `private` só podem ser acessados diretamente pela própria classe base.

No seu exercício, qual classe representa o tipo geral e qual representa uma especialização dele?

qual a diferença entre virtual e override?

virtual fica no método da **classe base**: permite que uma classe filha mude seu comportamento.

override fica no método da **classe filha**: indica que ela está substituindo esse comportamento.

</> C#

```
class pessoa
{
    public virtual void apresentar()
    {
        Console.WriteLine("sou uma pessoa");
    }
}

class aluno : pessoa
{
    public override void apresentar()
    {
        Console.WriteLine("sou um aluno");
    }
}
```

Mesmo se você fizer:

</> C#

```
pessoa p = new aluno();
p.apresentar();
```

Ele imprime `sou um aluno` , porque o objeto criado é um `aluno` . Isso é **polimorfismo**.

Pensa no seu exercício: qual método tem um comportamento geral, mas precisa funcionar de maneira diferente em cada classe filha?

como adiciono um objeto recebido por parametro em uma lista?

Você usa o método **Add()** da lista, passando o parâmetro recebido:

</> C#

```
void adicionar(pessoa novaPessoa)
{
    pessoas.Add(novaPessoa);
}
```

Aqui:

- `novaPessoa` é o objeto recebido por parâmetro.
- `pessoas` é uma lista do tipo `List<pessoa>` , que precisa estar inicializada.
- `Add()` coloca esse objeto no final da lista.

Você não precisa criar outro objeto com `new` : o parâmetro já recebeu um objeto.

Qual é o tipo dos objetos que sua lista vai guardar?

como o `foreach` sabe qual versão de uma função executar?

Quem decide isso **não é o `foreach`**, é o **polimorfismo**. O `foreach` apenas pega um objeto da lista por vez.

Imagine uma `List<Pessoa>` contendo um `aluno` e um `professor` :

</> C#

```
foreach (Pessoa p in pessoas)
{
    p.apresentar();
}
```

Se `apresentar()` for `virtual` na classe `Pessoa` e tiver `override` nas filhas:

- Quando `p` aponta para um `aluno`, executa a versão de `aluno`.
- Quando aponta para um `professor`, executa a versão de `professor`.

A variável é do tipo `Pessoa`, mas o objeto continua sendo um `aluno` ou um `professor`.

E se uma classe filha não fizer `override`, qual versão você acha que será executada?

como guardo objetos de classes diferentes em uma lista da classe `Pai`?

Você declara a lista com o **tipo da classe pai** e adiciona objetos das classes que herdam dela:

</> C#

```
List<Pessoa> pessoas = new List<Pessoa>();

pessoas.Add(new aluno());
pessoas.Add(new professor());
```

Isso funciona se `aluno` e `professor` herdarem de `Pessoa` : **um aluno é uma pessoa, e um professor também.**

Os objetos continuam sendo das respectivas classes. Por isso, ao chamar um método `virtual`, o `override` de cada um pode ser executado.

Só um detalhe: esses `new aluno()` e `new professor()` pressupõem construtores sem parâmetros. Se seus construtores recebem dados, você precisa passá-los entre os parênteses.

qual a diferença entre herdar de uma classe e ter uma lista de objetos dela?

Herdar significa "é um". Ter uma lista significa "tem vários".

Por exemplo:

- `aluno` herda de `pessoa` : um aluno **é uma pessoa**.
- `turma` tem uma `List<aluno>` : uma turma **tem vários alunos**.

Na herança, a classe filha recebe os membros da classe base, respeitando as regras de acesso, e pode personalizar métodos com `override` .

Ao ter uma lista, a classe apenas guarda objetos e pode chamar seus métodos. **Ela não passa a ser daquele tipo nem herda seus membros.**

Para decidir no exercício, pense: **essa classe é uma versão especializada da outra ou é um grupo que reúne objetos dela?**

oq seria o polimorfismo em si?

Polimorfismo é tratar objetos de tipos diferentes por um tipo comum, permitindo que cada um tenha seu próprio comportamento.

Pensa no exemplo: `aluno` e `professor` herdam de `pessoa` . Você pode guardar ambos numa lista de pessoas e chamar:

</> C#

```
p.apresentar();
```

A chamada é igual, mas o comportamento depende do objeto:

- Um `aluno` executa a apresentação de aluno.
- Um `professor` executa a apresentação de professor.

Nesse caso, isso funciona com **`virtual` na classe pai e `override` nas filhas**.

Ou seja: você pode pedir **"se apresente"** a todas as pessoas, sem precisar perguntar antes se cada uma é aluno ou professor. Cada objeto sabe como responder 😊